Ministerul Educației și Cercetării al Republicii Moldova
Universitatea Tehnică a Moldovei
Facultatea Calculatoare, Informatică și Microelectronică

Raport
la
Lucrarea de laborator Nr. 11

Disciplina: Tehnici de programare

Tema: ” Obiecte și clase”

A efectuat : st. gr.SI – 261 Talchiv Corina

A verificat : asistent universitar Dumitrașcu Marius

Chișinău – 2026

SARCINA LUCRĂRII

Sarcina laboratorului este să modelezi în Python procese și evenimente Windows ca obiecte, nu ca dicționare: definești clasele Proces și Eveniment cu @dataclass, fiecare cu o metodă e_suspect() care decide singură dacă obiectul e suspect (un proces care rulează din Temp sau Roaming, un eveniment cu ID 4625, 1102, 7045 sau nivel critical), le încarci din fișiere JSON cu Proces(**d), apoi creezi clasa-container Inventar (cu constructorul din_json, metodele suspecte() și pe_user()), scrii un main() cu argparse care generează un raport de alarme în text sau JSON (cu asdict), adaugi teste cu pytest pentru e_suspect, și predai totul prin git în fișierele inventar.py și test_inventar.py.

Scopul lucrării

Scopul lucrării este modelarea datelor unui sistem Windows (procese și evenimente de securitate) sub formă de **obiecte**, în loc de dicționare separate. Se urmărește:

- definirea claselor cu decoratorul @dataclass;
- adăugarea de **metode** care permit obiectului să "răspundă singur" (e_suspect);
- încărcarea obiectelor din fișiere JSON;
- crearea unei clase-container Inventar cu metode de interogare;
- generarea unui raport de alarme în terminal sau în format JSON;
- verificarea codului cu teste automate (pytest).

Contextul este unul de securitate informațională: procesele care rulează din foldere ale utilizatorului (Temp, Roaming) și evenimentele Windows cu ID-uri critice sunt indicii clasice de activitate malițioasă.

Lucrarea A. Procesul și evenimentul ca obiecte

Constante

```
from dataclasses import dataclass, asdict
import json

CAI_SUSPECTE = ("\\Temp\\", "\\Roaming\\")
SUSPECTE_ID = {4625, 1102, 7045}
```

CAI_SUSPECTE este un tuplu cu fragmente de cale considerate suspecte. Backslash-ul se scrie dublu (\\) pentru că în Python \ este caracter de escape.

SUSPECTE_ID este o mulțime (set) cu ID-uri de evenimente Windows cu semnificație de securitate:

4625: autentificare eșuată (posibil atac brute force);

1102: jurnalul de audit a fost șters (urmă tipică de acoperire a activității);

7045: a fost instalat un serviciu nou (metodă de persistență).

Mulțimea permite verificarea rapidă a apartenenței cu in.

Clasa Proces

```
@dataclass
class Proces:
    pid: int
    nume: str
    user: str
    cale: str
```

```
    def e_suspect(self):
        return any(c in self.cale for c in CAI_SUSPECTE)
```

Ce face: descrie un proces Windows prin patru câmpuri legate (identificator, nume, utilizator, calea executabilului). Decoratorul @dataclass generează constructorul și afișarea, deci print(p) arată toate câmpurile, nu o adresă de memorie.

Metoda e_suspect: obiectul se uită la propria cale (self.cale) și răspunde True dacă ea conține oricare dintre fragmentele din CAI_SUSPECTE. Funcția any(...) primește un generator și se oprește la prima potrivire.

Clasa Eveniment

```
@dataclass
class Eveniment:
    id: int
    nivel: str
    sursa: str
    cont: str
    mesaj: str

    def e_suspect(self):
        return self.id in SUSPECTE_ID or self.nivel == "critical"
```

Ce face: modelează o intrare din jurnalul de securitate. Metoda e_suspect întoarce True dacă ID-ul evenimentului este în SUSPECTE_ID **sau** dacă nivelul este "critical".

Ambele clase au o metodă cu același nume, dar cu logică diferită. Fiecare obiect știe singur ce înseamnă "suspect" pentru tipul lui.

Funcțiile de încărcare din JSON

```
def procese_din_json(cale):
    with open(cale, encoding="utf-8") as f:
        return [Proces(**d) for d in json.load(f)]

def evenimente_din_json(cale):
    with open(cale, encoding="utf-8") as f:
        return [Eveniment(**d) for d in json.load(f)]
```

Pas cu pas:

1. open(cale, encoding="utf-8") deschide fișierul; with îl închide automat la final.
2. json.load(f) citește JSON-ul și întoarce o listă de dicționare.
3. Proces(**d) despachetează fiecare dicționar în câmpurile obiectului, pe nume.
4. List comprehension-ul construiește lista finală de obiecte.

Astfel, datele brute (dicționare) devin obiecte cu comportament (metode).

Lucrarea B. Clasa Inventar și raportul

Clasa Inventar

```
@dataclass
class Inventar:
    procese: list

    @classmethod
    def din_json(cls, cale):
        with open(cale, encoding="utf-8") as f:
            return cls([Proces(**d) for d in json.load(f)])

    def __len__(self):
        return len(self.procese)

    def suspecte(self):
        return [p for p in self.procese if p.e_suspect()]

    def pe_user(self):
        grupe = {}
        for p in self.procese:
            grupe.setdefault(p.user, []).append(p.nume)
        return grupe
```

`Inventar` este o **clasă-container**: ține lista de obiecte `Proces` și oferă metode pentru a o interoga.

`procese: list` - Câmpul unic: lista obiectelor `Proces`.

`din_json(cls, cale)` - **Constructor alternativ.** Fiind `@classmethod`, primește clasa `cls` și întoarce un `Inventar` gata umplut din fișier. Se apelează `Inventar.din_json(...)`, fără a avea deja un obiect.

`__len__` - Permite `len(inv)`; întoarce numărul de procese.

`suspecte()` - Filtrează procesele care se declară suspecte, deci **delegă** decizia metodei `e_suspect` a fiecărui `Proces`.

`pe_user()` - Grupează numele proceselor pe utilizator, într-un dicționar de liste. `setdefault(cheie, [])` creează lista dacă cheia nu există, apoi `append` adaugă numele.

Testele - Fișier: arsenal/windows/test_inventar.py

```
from arsenal.windows.inventar import Proces

def test_proces_din_temp_e_suspect():
    p = Proces(1, "x.exe", "ana", "C:\\Users\\ana\\AppData\\Local\\Temp\\x.exe")
    assert p.e_suspect() is True

def test_proces_din_windows_nu_e_suspect():
    p = Proces(2, "svchost.exe", "SYSTEM", "C:\\Windows\\System32\\svchost.exe")
    assert p.e_suspect() is False
```

Rularea și rezultatele:

```
(.venv) PS D:\Securitatea_Informationala_UTM\Tehnici_de_Programare\tehnici-prog-talchiv_C\L11_TP> python -m arsenal.windows.inventar arsenal/windows/start
Procese: 7 | suspecte: 2
  [3120] update.exe  (ana)  C:\Users\ana\AppData\Local\Temp\update.exe
  [3890] svch0st.exe  (ana)  C:\Users\ana\AppData\Roaming\svch0st.exe
Evenimente: 8 | alarma: 5
  4625 [warning] administrator: An account failed to log on
  4625 [warning] administrator: An account failed to log on
  4625 [warning] administrator: An account failed to log on
  7045 [warning] SYSTEM: A service was installed: WinUpdateSvc
  1102 [critical] administrator: The audit log was cleared
(.venv) PS D:\Securitatea_Informationala_UTM\Tehnici_de_Programare\tehnici-prog-talchiv_C\L11_TP> python -m pytest -q arsenal/windows/test_inventar.py
..                                                                       [100%]
2 passed in 0.05s
(.venv) PS D:\Securitatea_Informationala_UTM\Tehnici_de_Programare\tehnici-prog-talchiv_C\L11_TP> python -m arsenal.windows.inventar arsenal/windows/start --json
{
  "suspecte": [
    {
      "pid": 3120,
      "nume": "update.exe",
      "user": "ana",
      "cale": "C:\\Users\\ana\\AppData\\Local\\Temp\\update.exe"
    },
    {
      "pid": 3890,
      "nume": "svch0st.exe",
      "user": "ana",
      "cale": "C:\\Users\\ana\\AppData\\Roaming\\svch0st.exe"
    }
  ],
  "alarma": [
    {
      "id": 4625,
      "nivel": "warning",
      "sursa": "Security",
```

Figură 1

```
      "nivel": "warning",
      "sursa": "Security",
      "cont": "administrator",
      "mesaj": "An account failed to log on"
    },
    {
      "id": 4625,
      "nivel": "warning",
      "sursa": "Security",
      "cont": "administrator",
      "mesaj": "An account failed to log on"
    },
    {
      "id": 4625,
      "nivel": "warning",
      "sursa": "Security",
      "cont": "administrator",
      "mesaj": "An account failed to log on"
    },
    {
      "id": 7045,
      "nivel": "warning",
      "sursa": "System",
      "cont": "SYSTEM",
      "mesaj": "A service was installed: WinUpdateSvc"
    },
    {
      "id": 1102,
      "nivel": "critical",
      "sursa": "Security",
      "cont": "administrator",
      "mesaj": "The audit log was cleared"
    }
  ]
```

Figură 2

CONCLUZII

În această lucrare s-au modelat procesele și evenimentele unui sistem Windows ca obiecte Python. Clasele `Proces` și `Eveniment` combină datele cu logica de detecție (`e_suspect`), iar clasa `Inventar` grupează obiectele și oferă interogări (`suspecte`, `pe_user`). Încărcarea din JSON cu `**d` și scrierea inversă cu `asdict` leagă obiectele de formatul de schimb de date, iar `argparse` transformă programul într-un instrument de linie de comandă. Testele cu `pytest` confirmă corectitudinea criteriului de detecție.

Abordarea pe obiecte face codul mai ordonat, mai ușor de extins (de exemplu cu `explica()` sau `scor()`) și apropiat de modul real în care se construiesc instrumentele de analiză în securitatea informațională.